3-4 · DuckDB를 이용한 데이터 분석 · 이론 1H

SQL 분석 문법 — 집계·윈도 함수·CTE

2-2 의 문법이 파일에 그대로 — QUALIFY 한 줄 더

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① count(*)·count(열)·count(DISTINCT 열)의 차이를 말한다

② 윈도 함수가 행을 줄이지 않는 이유를 설명한다

③ QUALIFY 가 CTE 한 겹을 대신하는 지점을 안다

④ .df() 를 언제 불러야 하는지 판단한다

다음 1.5H 실습에서 파일끼리 JOIN 하고 Pandas 답과 교차 검증합니다

MySQL(2-2) 과 DuckDB — 같은 것, 더해진 것

	MySQL 9.7 (2-2)	DuckDB (오늘)
CTE	WITH … AS (…)	같음
Window	OVER (PARTITION BY …)	같음
순위 거르기	CTE로 감싸고 WHERE rn = 1	QUALIFY rn = 1 (한 줄로)
GROUP BY 축약	GROUP BY 1, 2 가능	같음 + GROUP BY ALL
SELECT 제외	없음	SELECT * EXCLUDE (열)
대상	적재된 테이블	파일·DataFrame 을 그대로

GROUP BY 는 접고 Window 는 붙인다 · OVER 안의 ORDER BY 가 누적을 만든다 — 2-2 도해 두 개를 다시 보고 오세요

집계 함수 다시 보기 — count 세 가지와 avg 의 NULL

1. **GROUP BY 는 접는다**
   그룹마다 하나의 값으로 요약 — 결과 행 수가 그룹 수만큼 줄어든다
2. **count(*) 와 count(열)**
   count(열)은 그 열이 NULL 인 행을 세지 않는다
3. **count(DISTINCT 열)**
   고유값 개수 — 조인 후 부풀려진 건수를 바로잡을 때(2-4에서 본 문제)
4. **avg 는 NULL 을 빼고 계산한다**
   「몇 건을 빼고 낸 평균인지」 함께 보는 습관

윈도 함수 — 행을 줄이지 않고 옆에 붙인다

1. **PARTITION BY — 「반」을 나눈다**
   PARTITION BY region 이면 지역마다 따로 순위를 센다

2. **ORDER BY — 창 안의 순서**
   순서를 정하고 동시에 누적 범위를 만든다(2-2 두 번째 도해)

3. **ROWS BETWEEN — 창 범위 지정**
   이동 평균에 쓴다 — 6 PRECEDING AND CURRENT ROW

비유

전교 등수 아닌
반 안의 등수

> **rank·dense_rank·row_number 는 동률 처리가 다르다 — 「정확히 N개」면 row_number, 「공동 순위 모두」면 rank**

4. QUALIFY

순위 거르기 — CTE 한 겹 vs QUALIFY 한 줄

CTE 방식 — 어디서나 통함

WHERE 에 윈도 함수를 못 쓰니 한 겹 감싼다

WITH ranked AS → **SELECT * FROM ranked**

rank() OVER (...) AS rnk

WHERE rnk <= 3

VS

QUALIFY — DuckDB 방식

같은 질의를 한 줄로 — 표준 SQL 은 아니다

rank() OVER (...) AS rnk → **QUALIFY rnk <= 3**

PARTITION BY region

감싸지 않고 바로

5. CTE

CTE — 긴 질의에 이름표를 붙여 단계로

01

읽고 고치기 쉽다

「1. 육수 만들기, 2. 면 삶기」처럼
한 덩어리 중첩 질의보다 낫다

02

디버깅에 유리하다

각 CTE 를 따로 SELECT 해
중간 결과를 확인할 수 있다

QUALIFY 는 DuckDB·Snowflake·BigQuery 에서만 된다 — 옮겨 다닐 코드라면 CTE 방식이 안전

DataFrame 왕복 — .df() 와 FROM 변수이름

1. **SQL → DataFrame**
   duckdb.sql("SELECT … FROM 'events.parquet'").df()

2. **DataFrame → SQL**
   파이썬 변수 df 를 그대로 FROM df 에 쓴다

3. **다른 형태로도**
   .fetchall() 파이썬 리스트 · .to_arrow_table() PyArrow 테이블

.df() 의 뜻

결과 전부를 메모리에

> **집계 전 원본에 부르면 100만 행이 통째로 올라온다 — 반드시 줄인 다음에 부른다**

7. 역할 나누기

언제 SQL, 언제 Pandas — 무거운 것과 세밀한 것

작업	어디서	왜
수백만 행 필터·집계·조인	SQL (DuckDB)	읽는 양을 줄이고 메모리를 아낀다
그룹별 순위·누적	SQL	윈도 함수가 간결하다
구간 나누기·라벨 붙이기	Pandas (pd.cut)	경계 조정이 쉽다
정규식·문자열 세밀 처리	Pandas	함수가 풍부하다
시각화·머신러닝 입력	Pandas	라이브러리가 DataFrame 을 받는다
결과를 파일로	둘 다	COPY 또는 to_parquet

무거운 것은 SQL 로 줄이고, 세밀한 것은 DataFrame 으로 다듬는다 — 3-5 하이브리드의 밑그림

정리

이 1시간에서 꼭 가져갈 것

1. 2-2 의 CTE·Window 문법이 DuckDB 에서 그대로 통한다
2. QUALIFY 는 편하지만 DuckDB 계열에서만 된다
3. CTE 는 읽기 쉽게 만들 뿐 아니라 디버깅에도 유리하다
4. .df() 는 반드시 결과를 줄인 다음에 부른다
5. 무거운 것은 SQL, 세밀한 것은 Pandas

3-4 · 이론 1H 마무리

다음은 SQL 집계·조인 분석 실습 (1.5H)

Parquet 와 CSV 를 적재 없이 곧바로 JOIN 하고, 3-2 의 Pandas 결과와 교차 검증합니다

- ✓ **상위 지출 사용자 — GROUP BY · LIMIT**
- ✓ **파일끼리 JOIN 해 지역별 매출**
- ✓ **CTE + 윈도 함수로 누적 매출·상위 3명**
- ✓ **이동 평균 — Pandas rolling 과 비교**

준비할 것

3-1·3-2·3-3 의 파일 넷